Great—this is exactly the kind of explanation interviewers love. Let's walk through a **step-by-step memory diagram simulation** of the **tri-color mark-and-sweep algorithm**, including a mutation case.

🧩 Initial Heap Setup

Assume this object graph:

```
Roots → A
A → B, C
B → D
C → (nothing)
D → (nothing)
E → F    (unreachable island)
F → (nothing)
```

👉 Objects: **A, B, C, D, E, F**

🎨 Step 0: Initialization

All objects start **white**, except roots:

```
Gray:   A
White:  B, C, D, E, F
Black:  -
```

🔄 Step 1: Process A

- A is gray → scan its children: **B, C**
- Turn B and C → gray
- A → black

```
Gray:   B, C
White:  D, E, F
Black:  A
```

🔄 Step 2: Process B

- B is gray → scan child: **D**
- D → gray
- B → black

```
Gray:   C, D
White:  E, F
Black:  A, B
```

🔄 Step 3: Process C

- C has no children
- C → black

```
Gray:   D
White:  E, F
Black:  A, B, C
```

🔄 Step 4: Process D

- D has no children
- D → black

```
Gray:   -
White:  E, F
Black:  A, B, C, D
```

🧹 Step 5: Sweep Phase

- White objects = **E, F**
- These are **garbage** → **reclaimed**

⚠️ Now the Interesting Part: Mutation During GC

Let's simulate a **concurrent mutation** (this is where interviews get tricky).

🧨 Scenario: Mutation Happens Mid-GC

At this moment:

```
Gray:   C
White:  D, E, F
Black:  A, B
```

Now suppose the application does:

```
A → D    (new reference added)
```

👉 But:

- A is already **black**
- D is still **white**

❗ Problem: Broken Invariant

We now have:

```
Black (A) → White (D)
```

This violates the rule:

> ❌ No black object should point to a white object

👉 If GC continues:

- D might never be visited
- It gets incorrectly collected → **BUG**

🛠️ Fix Using Write Barrier (Dijkstra)

When this assignment happens:

```
A → D
```

We intercept it:

👉 If **black** ⟶ **white**, then:

```
D → Gray
```

✅ Corrected State

```
Gray:   C, D
White:  E, F
Black:  A, B
```

Now D will be processed safely.

🔄 Continue Execution

Process C:

```
Gray:   D
Black:  A, B, C
```

Process D:

```
Gray:   -
Black:  A, B, C, D
```

🧹 Final Sweep

```
White: E, F → collected
```

🧠 Key Takeaways (Interview Gold)

1. Tri-color = Graph traversal

- White → unvisited
- Gray → discovered
- Black → fully processed

---

2. Critical invariant

> **No black → white references**

---

3. Why write barriers?

- App mutates graph during GC
- Barriers **preserve correctness**

---

4. Two mental models

Without mutation:

👉 Simple BFS/DFS

With mutation:

👉 BFS + "self-healing" using write barriers

🔥 One-liner for Interviews

> "Tri-color marking maintains a no black-to-white invariant. Write barriers ensure that any mutation during concurrent marking doesn't violate reachability guarantees."

If you want, I can next show:

- **SATB vs Incremental Update (used in G1 vs CMS)**

- Or **how this maps to JVM G1 phases step-by-step** (very commonly asked)